FAQ

SWAT+ Documentation

docs.swat.tamu.edu

Contents

Purpose . . . 1
Pages . . . 1
Checks after every run . . . 2
Purpose . . . 2
Message locations . . . 2
Errors that stop the run . . . 2
Messages in diagnostics.out . . . 4
Names matched without a message . . . 5
Files the model creates . . . 6
Old-format inputs . . . 6
Runs that stop without a message . . . 6
Related files . . . 7
Purpose . . . 7
Balance check . . . 7
Precipitation . . . 7
Surface runoff . . . 8
Evapotranspiration . . . 8
Management . . . 8
Percolation . . . 8
Lateral and tile flow . . . 8
Basin totals . . . 9
Related files . . . 9
Purpose . . . 9
One run per processor core . . . 9
Build type . . . 10
Time step . . . 10
Output volume . . . 10
Object count and storage . . . 11
Summary . . . 11

Purpose

These pages help when a SWAT+ run stops, completes with results that do not make sense, or takes too long. First runs and installation problems are covered in Getting started, and adjusting parameters to observations in Calibration.

Pages

- Common runtime errors: the messages SWAT+ prints when a run stops or a name in the inputs is not matched, the names it matches without a message, files it creates when they are missing, and inputs in an older format.
- Water balance issues: implausible precipitation, runoff, ET, percolation, lateral and tile flow, and basin totals that do not match the HRUs.

- Performance tips: build type, time step, output volume and the size of the carbon output files.

Checks after every run

1. `simulation.out` ends with `Execution successfully completed`, and `success.fin` exists. If not, the run stopped early: keep the screen output, since stop messages are not copied to the log files.
2. Read `diagnostics.out` to the end. Every line after the first is a name that was not matched, and SWAT+ used a default or left the feature off.
3. Check the names that are not reported when they do not match, above all the names in `hru-data.hru`, which fall back to the first record of each file. See Names matched without a message.
4. Read `checker.out`: the soil group, rooting depth, field capacity, `perco`, `latq_co` and tile flag of each HRU show whether the intended soil and parameters were used.
5. Compare the average annual water balance with regional estimates before calibrating.

The file layouts are in Checker and diagnostics. The revision of the executable is in the banner at the top of `simulation.out` (`Revision 62.0.1`); inputs written for an earlier revision can be misread without a message (see Old-format inputs).

Sources

- `diagnostics.out` and `files_out.out` are opened in `proc_bsn.f90`; the banner and the completion message are written by `main.f90.in`.
- Name lookup for HRUs: `hru_read.f90`.

Purpose

This page lists the messages SWAT+ 62.0.1 writes when a run stops or when it cannot match a name in the inputs, and the input problems that change the results without any message. For each one it gives the cause and the fix.

Message locations

Place	Content
the screen (standard output)	errors that stop the run, written just before it stops
`diagnostics.out`	names that could not be matched while the inputs were read; the run continues
`simulation.out`	the revision banner and one line per simulated day, ending with `Execution successfully completed`
`gwflow_record`	messages from the gwflow module, including the ones that stop the run

Messages printed on the screen are not copied to `simulation.out`. When the model is started by a script or another program, keep the screen output, for example with `swatplus > run.log 2>&1`. The layout of the log files is described in Checker and diagnostics.

In the messages below, `<name>` stands for a name from the inputs and `<n>` for a number. SWAT+ pads names with spaces, so the gaps in a real message are wider.

Errors that stop the run

Missing object.cnt

```
Cannot find object.cnt input file
```

The file named on the `simulation` line of `file.cio` (normally `object.cnt`) does not exist, or the name is `null`. Usually the model was started outside the project folder. Change into the folder that contains `file.cio` and start it again. Started in an empty folder, SWAT+ also leaves a few files behind (see Files the model creates).

Carbon input files

With `carbon = 2` in codes.bsn, two files are required. SWAT+ stops with one of these messages, on the screen and in `diagnostics.out`:

```
ERROR: <name> is required when carbon is enabled (codes.bsn carbon = 2)
ERROR: <name> could not be opened
ERROR: <name> data/values line is missing or could not be parsed (expected 28 values)
ERROR: <name> provided <n> row(s); expected <n>
```

The first file is the one named in the third slot of the `basin` line of file.cio, normally carbon.bsn. The second has no slot: its name is the first name with `_lyr` added before the extension, normally carbon_lyr.bsn. Provide carbon.bsn with its 28 values and carbon_lyr.bsn with a row for each layer group, or set `carbon` to 0. The old carb_coefs.cbn is no longer read.

If the message names a file that is not a carbon file, such as `ERROR: climate is required when carbon is enabled`, the `basin` line of file.cio has only two names. See Old-format inputs.

print.prt rows

```
Error: The output object <name> in the input file print.prt is not a valid object.
       and cannot be processed
Error: <name> print object is duplicated in the input file print.prt.  Aborting
```

These checks apply when `use_obj_lbls` is `y` in print.prt, so that the rows are read by name. A row name that SWAT+ does not know, or a row given twice, stops the run. The valid names, and the files each one opens, are listed in print.prt rows and files.

Output folder

```
! ERROR: cannot create path specified in file.cio (out_path)
  > Windows-style path detected on Unix system: <name>
! ERROR: cannot create or access path specified in file.cio (out_path)
  > Path: <name>
```

The optional output folder line of file.cio names a folder SWAT+ cannot use. On Linux and macOS a folder name that contains `:` is taken as a Windows path and refused. The second message means the folder could not be created or opened, often because of file permissions. See Output folder.

gwflow input files

The gwflow module writes its messages to `gwflow_record` in the run folder, so the screen shows only that the program stopped. These messages stop the run:

Message in `gwflow_record`	Cause
`ERROR: gwflow.cells row has too few columns at i=<n>`	row *n* of cells.gw has fewer than the 14 required columns
`ERROR: gwflow.cells row out of order at i=<n> cell_id=<n>`	the cell IDs in cells.gw are not 1, 2, 3, … in file order
`ERROR: gwflow.cellcons row has too few columns at i=<n>`	row *n* of cellcon.gw has fewer than 2 columns
`ERROR: gwflow.cellcons row out of order at i=<n> cell_id=<n>`	the cell IDs in cellcon.gw are not in file order
`neither gwflow.hrucell or gwflow.lsucell found; stop simulation`	`conn_type` in codes.gw is neither 1 nor 2

The messages use older file names: `gwflow.cells` is cells.gw and `gwflow.cellcons` is cellcon.gw. With `conn_type` 1 or 2 and neither hrucell.gw nor lsucell.gw present, the run does not stop. See gwflow inputs.

Fortran run-time errors

Some readers do not check what they read. A missing line, a missing file or text where a number is expected then stops the run with a message from the Fortran run-time library instead of SWAT+. With gfortran builds it contains `Fortran runtime error`, and with Intel builds it starts with `forrtl: severe`. The message usually gives the file name or the unit number. Check that file for a missing or extra line or a wrong number of columns, and compare it with its page in the Input Reference.

Most readers do check, and do not stop. When a value in a row cannot be read, they stop reading that row without a message, and the columns after it keep their defaults. A row with a stray character or a missing value can therefore change the results silently.

Messages in diagnostics.out

Line 1 of `diagnostics.out` is `DIAGNOSTICS.OUT FILE`. Each later line is written while the inputs are read, and the run continues. From a run of the `Ames_sub1` reference dataset:

```
agrr_lum                                  AGRR_rot                                  not found in
↪  management.sch
ames.tem                                            file not found (tgage)
```

Land use names

Written for a row of `landuse.lum` that names a record that is not found. The line gives the land use name, the missing name and one of these endings:

Ending	Column	Result
`not found in plant.ini`	`plnt_com`	no plants at the start
`not found in management.sch`	`mgt`	no scheduled operations: no planting, fertilizer or harvest
`not found in cntable.lum`	`cn2`	built-in curve numbers 30, 55, 70 and 77 for soil groups A to D
`not found (landuse-consprac)`	`cons_prac`	P factor 1.0
`not found (landuse-tiledr)`	`tile`	no tile drain
`not found (landuse-septic)`	`sep`	no septic system
`not found (landuse-fstrip)`	`vfs`	no filter strip
`not found (landuse-grassww)`	`grww`	no grassed waterway
`not found (landuse-bmpuser)`	`bmp`	no user BMP

Correct the name in `landuse.lum`, or add the missing record to the file it points to. See Name matching. `Ames_sub1` writes four `not found in management.sch` lines, so the HRUs with those land uses have no management.

Plant names

```
mgt schedule <n> op numb <n> <name> not found in plants.plt database
plant com <n> plant numb <n> <name> not found in plants.plt database
```

The first is written for a `plnt` line of `management.sch` whose plant name is not in `plants.plt`; the numbers are the position of the schedule in the file and of the line in the schedule. The second is written for a plant in `plant.ini` that is not in `plants.plt`. Plant names are case-sensitive.

Weather station names

```
<name> file not found (tgage)
```

A station in `weather-sta.cli` names a data file that is not listed in the matching index file. The ending gives the column: `(pgage)` precipitation and `pcp.cli`, `(tgage)` temperature and `tmp.cli`, `(sgage)` solar radiation, `(hgage)` humidity, `(wgage)` wind, `(petgage)` PET and `(atmodep)` atmospheric deposition. For the five weather variables,

SWAT+ then generates that variable for the station on every day from its weather generator record. In `Ames_sub1`, weather-sta.cli names `ames.tem` while tmp.cli lists `ames.tmp`, so the temperatures are generated.

`file not found (wgn)` means the `wgn` name of the station is not in weather-wgn.cli. 62.0.1 has no record to fall back on in that case, and the generated weather of that station is undefined. Correct the name before running. See Unmatched names.

Reservoir, wetland and other names

Ending	Name that was not found
`not found (res-init)`, `(res-hyd)`, `(res-release)`, `(res-sed)`, `(res-nut)`	a record named in reservoir.res
`not found (wet-init)`, `(wet-hyd)`, `(wet-release)`, `(wet-sed)`, `(wet-nut)`	a record named in wetland.wet
`not found (wetland.wet)`	the `surf_stor` name of an HRU in hru-data.hru; the HRU has no surface storage
`not found (field.fld)`	the `field` name of an HRU; the HRU uses the built-in field size
`not found (plants.plt)`	the plant of an HRU in hru-lte.hru
`entry in (hru-lte.hru) not found in (lum.dtl)`	a decision table named in hru-lte.hru
`not found (initial.cha)`, `(hydrology.cha)`, `(sediment.cha)`, `(nutrients.cha)`	a record named in the legacy channel.cha, which 62.0.1 reads but does not route

Carbon layer files

```
<name>: layer id <n> is out of range (1 to <n>); row ignored
carbon_layers.prt: n_layers must be >= 1; got <n>; default <n> kept
```

The first means a row of carbon_lyr.bsn has a layer number other than 1 or 2, and the row is skipped. The second means the optional layer count file has a value below 1, and the per-layer carbon outputs use the largest layer count of any HRU.

Names matched without a message

Several names are looked up without any report when they are not found. The run continues with the result in the table, so check these names by hand or in SWAT+ Editor.

Name	Result when it is not found
`topo`, `hydro`, `soil`, `lu_mgt`, `soil_plant_init` or `snow` in hru-data.hru	the HRU uses the first record of the target file. The reader has messages for these names, but they are never written in 62.0.1.
`wst` in a connection file such as hru.con	the object uses the first station in weather-sta.cli. The stations must also be listed in ascending order of name for the lookup to work. See Weather station lookup.
fertilizer name of a `fert` line in management.sch	nothing is applied. The name is read from `op_data1`; see the management.sch example.
harvest operation of a `harv` line	nothing is harvested
tillage name of a `till` line	no soil is mixed
transplant name of a `plnt` line or a decision table plant action	the plant starts from seed. In 62.0.1 the file is transplant.ops, and a file named `transplant.plt` is not read. See transplant.ops.
manure name of a `manu` line	the manure is found by name in manure_db.frt, but its make-up comes from the row of manure_om.frt in the same position, whatever `org_min` names. See manure_om.frt.

Name	Result when it is not found
decision table named on an automatic operation line of `management.sch`	the table is not applied. See Automatic operations.
`urban` or `ov_mann` in `landuse.lum`	the HRU is not urban, or keeps Manning's n of 0.05

Files the model creates

For a number of input files, 62.0.1 opens the named file without checking that it exists, and the operating system then creates an empty file of that name. The run continues as if the file had no records. This happens for `time.sim` and `print.prt`, the place-of-use and point-of-diversion files of water allocation, `wtps_wuses.wal`, and the initial-condition files for pesticides, pathogens, metals, salts and constituents (`pest_hru.ini`, `salt_hru.ini`, `cs_aqu.ini` and others). For some of these slots a `null` name creates a file named `null`.

An empty file left behind changes the next run: an empty `time.sim`, for example, leaves the run without a valid simulation period. After a run that misbehaves, list the empty files in the project folder (`find . -size 0` on Linux and macOS) and restore or delete them. If in doubt, write the inputs again from SWAT+ Editor.

Old-format inputs

Files written for an earlier revision are read without a version check. These changes matter most for 62.0.1:

- **file.cio `basin` line.** It now has three names, the third being the carbon file. With only two, every later line of `file.cio` is read one line too low and all later file names are wrong, with no message. With `carbon = 2` the run stops with `ERROR: climate is required when carbon is enabled`. See Older file.cio files.
- **file.cio `water_rights` line.** It now has two names. An old line with `water_allocation.wro element.wro water_rights.wro` makes SWAT+ read `water_allocation.wro` as the place-of-use file.
- **transplant.plt.** Renamed to `transplant.ops`. A project that still has `transplant.plt` plants from seed.
- **pesticide.pes.** Three columns were added after the existing 13 values. Older rows are still read, and the new columns keep their defaults, which means no plant uptake. See pesticide.pes.
- **print.prt.** A file with `dbout` in place of `use_obj_lbls` is read by position, so an extra or missing row moves the switches of every later row to the wrong output group. See print.prt.
- **carb_coefs.cbn.** No longer read. Its settings are in `carbon.bsn` and `carbon_lyr.bsn`.

Most input files start with a title line and a column header line, and SWAT+ skips both. A file that starts with the header loses its first data row, with no message. In the reference data, `tillage.til` of `Ames_sub1` and `puddle.ops` of `Osu_1hru` have no title line, so their first entries are not read; see tillage.til and puddle.ops.

Runs that stop without a message

Full disk. When the disk fills, the run stops with a Fortran run-time error about writing a file, or with no message at all, and the last output files are cut short. Other programs that use the same disk start to fail as well. Daily output is the usual cause: a full 46-year run of the `Ames_sub1` reference dataset writes about 9 GB, almost all of it the daily carbon files switched on by its `hru_cb` and `hru_cb_vars` rows. Check the free space before long runs (`df -h .` on Linux and macOS) and switch off the daily outputs you do not need; see Output volume.

The last day. The last daily line in `simulation.out` is the last day that finished, so the problem occurred on the next day. Climate records that are shorter than the simulation do not stop the run: SWAT+ generates the missing days (see Weather outside the observed record).

A run that never gets past its first day. Earlier revisions could repeat the operations of a management schedule without end when every operation had the same month and day. 62.0.1 runs each operation at most once per pass on a given day, so a run that hangs at the start with 62.0.1 has another cause. A debug build with run-time checks usually stops with a message that points to it.

Related files

- Checker and diagnostics: the layout of `simulation.out`, `diagnostics.out`, `checker.out` and `files_out.out`.
- Water balance issues: runs that complete but give implausible results.
- Install and Quick start with Ames_sub1: first runs, `Permission denied` and the start-up banner.

Sources

- Stops: `basin_read_objs.f90` (`object.cnt`), `carbon_bsn_read.f90`, `basin_print_codes_read.f90` and `print_prt_error` in `basin_module.f90`, `output_path_module.f90`, `gwflow_read.f90`
- `diagnostics.out` is opened as unit 9001 in `proc_bsn.f90`. Writers: `landuse_read.f90`, `read_mgtops.f90`, `readpcom.f90`, `cli_staread.f90`, `hru_read.f90`, `hru_lte_read.f90`, `res_read.f90`, `wet_initial.f90`, `ch_read.f90`, `carbon_layers_read.f90`
- Weather fallback: `climate_control.f90`, `cli_precip_control.f90`, `cli_bounds_check.f90`, `cli_wgnread.f90`
- Same-date guard: `mgt_operatn.f90`

Purpose

When a run completes but `basin_wb_aa.txt` or `hru_wb_aa.txt` looks wrong, the cause is usually in the climate inputs, the curve numbers, the management, the soil and hydrology parameters, or the way the basin totals are weighted. This page goes through the water balance term by term. Many of the causes leave no message, so the checks below name the file and column to look at.

Before changing parameters, read `diagnostics.out` and `checker.out` (see Checker and diagnostics) and the list of names matched without a message. An HRU that silently uses the first record of `landuse.lum`, `soils.sol` or `hydrology.hyd` gives a water balance that no parameter change will fix.

Balance check

For an HRU, landscape unit or the basin, the average annual water balance should close:

$$P + I + Q_{\text{runon}} \approx ET + WYLD + PERC + \Delta SW + \Delta SNO$$

where P is `precip`, I is `irr`, Q_{runon} is `surq_runon` + `latq_runon`, ET is `et`, $WYLD$ is `wateryld` (`surq_gen` + `latq` + `qtile`), $PERC$ is `perc`, and ΔSW and ΔSNO are the changes in soil water and snow over the period. The columns are described in Water balance outputs. A large gap points to wetland storage, gwflow exchange or water held in the surface runoff lag. A term that is far from what the climate and land cover suggest points to one of the sections below.

Precipitation

The `precip` column does not match the observed rainfall.

- **Weather station of the HRU.** Each HRU takes its station from the `wst` column of `hru.con`. A name that is not in `weather-sta.cli`, or a `weather-sta.cli` whose stations are not in ascending order of name, gives the HRU the first station in the file, with no message. See Weather station lookup.
- **Data file of the station.** A line `file not found (pgage)` in `diagnostics.out` means the station names a precipitation file that is not listed in `pcp.cli`, and SWAT+ generates the rainfall from the weather generator on every day. See Weather station names.
- **Simulation period.** Days before the start or after the end of the record, and days flagged as missing, are generated. See Weather outside the observed record.
- **Sub-daily runs.** With `step` above 1 in `time.sim`, a station with daily precipitation records gets no rainfall in its time steps. A sub-daily run needs sub-daily records or generated precipitation. See Time step.

`hru_pw` files and the `precip` column of `hru_wb` show what each HRU received, so compare a few HRUs with the station file before looking further.

Surface runoff

`surq_gen` is too high or too low.

- **Curve numbers.** The `cn2` column of `landuse.lum` names a record in `cntable.lum`, and the soil group in `soils.sol` picks one of its four values. A name that is not found is reported as `not found in cntable.lum`, and the HRU uses the built-in values 30, 55, 70 and 77 for groups A to D. The `hydgrp` column of `checker.out` shows the soil group each HRU uses.
- **Moisture adjustment.** `cn3_swf` in `hydrology.hyd` places the curve number for wet conditions between field capacity (0) and saturation (1). See Curve number soil water factor.
- **Runoff method.** `gampt` in `codes.bsn` selects the curve number method (0) or Green-Ampt (1), which is meant for sub-daily runs. See Runoff method.
- **Lag.** `surq_gen` is the runoff after the surface runoff lag and `surq_cont` the runoff generated on the day. A large difference between them in yearly totals means water is held in the lag.

Evapotranspiration

`et` is too high or too low.

- **PET method.** `pet` in `codes.bsn`: 0 Priestley-Taylor, 1 Penman-Monteith, 2 Hargreaves, 3 the PET file of the station. Penman-Monteith uses solar radiation, humidity and wind, which are generated when the station has no records for them. Compare the `pet` column of the water balance file with a regional estimate.
- **Compensation factors.** `esco` and `epco` in `hydrology.hyd` set how far down the profile soil evaporation and plant uptake can reach. `checker.out` lists the values each HRU uses. See Soil evaporation.
- **Plant cover.** If plants do not grow, `eplant` stays low and more water leaves as soil evaporation, runoff and percolation. Check biomass and LAI in the `hru_pw` files and yields in `crop_yld_aa.txt` (see Plant and weather).

Management

Crops do not grow, or grow without fertilizer or harvest.

- A land use whose `mgt` schedule is not in `management.sch` is reported as `not found in management.sch`, and its HRUs get no operations. The `Ames_sub1` reference dataset has four such land uses.
- Several operation names are not reported when they are not found: a fertilizer name gives no fertilizer, a harvest operation no harvest, a tillage name no mixing, and a transplant name a plant started from seed. In the `Ames_sub1` reference dataset, the fertilizer names of every `fert` line are in the wrong column, so the dataset applies no mineral fertilizer in 62.0.1. See the management.sch example.
- Set `mgtout` to `y` in `print.prt` to write `mgt_out.txt`, the log of every operation and decision table action that ran. An operation missing from the log did not happen.

Percolation

`perc` is close to zero.

`perco` in `hydrology.hyd` limits percolation out of the bottom of the soil profile, and the relation is strongly non-linear: 0.5 lets about 0.5 % of the potential percolation through, 0.9 about 3 %, and only values close to 1 allow free drainage. The value 0.5, used for every HRU in `Ames_sub1`, therefore leaves almost nothing to percolate. See Percolation from the soil profile.

Also check `sumfc` and `sumul` in `checker.out`, the water held at field capacity and saturation over the profile. Values that are the same for every HRU can mean the HRUs all use the first record of `soils.sol` because their soil names did not match.

Lateral and tile flow

`latq` or `qtile` is out of range.

- **Lateral flow.** `latq_co` and `lat_ttime` in `hydrology.hyd` scale and delay lateral flow. See Lateral flow.
- **Tile drains.** An HRU has a tile drain when the `tile` column of its land use names a record in `tiledrain.str`. The `tiledrain` column of `checker.out` shows 1 for those HRUs. A decision table `tile_install` action has no effect on an HRU whose land use had no tile record at the start. A tile `lag` of 0.01 h or less keeps tile water in the lag store, so `qtile` stays at zero. See Tile flow methods.
- **Septic systems.** In 62.0.1 a septic system adds no water or nutrients to the soil. See Septic simulation in 62.0.1.

Basin totals

The basin or landscape unit values do not match the HRU values.

Basin rows are sums of the HRU values weighted by the `BSN_FRAC` column of `ls_unit.ele`, and landscape unit rows by `RU_FRAC`. 62.0.1 uses these fractions as given, without scaling them to add up to 1. In the `Ames_sub1` reference dataset the 12 `BSN_FRAC` values add up to 2, so every basin water balance, nutrient balance, losses and plant file shows twice the area-weighted HRU average. Set each fraction to the HRU area divided by the basin area, and check that they add up to 1. The basin sum also assumes `ls_unit.ele` lists one element per HRU, in HRU order. See Aggregation and Nutrient balance and losses.

Routing unit hydrograph files (`ru_*`) add up every hydrograph type of every element: total flow plus the recharge, surface, lateral and tile parts. Their `flo` is therefore larger than the routing unit's outflow, which is in the `hydout` files. See ru files.

`flo` in the hydrograph files is a volume in m^3 per print step, although the units row gives m^3/s. Divide a daily value by 86,400 to compare it with a gauge record. See Hydrograph columns.

Related files

- Common runtime errors: messages and the names matched without a message.
- Water balance outputs: the columns and how they are aggregated.
- Ames_sub1 tutorial: a worked look at the reference dataset's water balance.
- Soft calibration: adjusting the water balance towards regional targets, once the inputs are right.

Sources

- Weather station and gage lookup: `cli_staread.f90`, `climate_control.f90`, `cli_precip_control.f90`
- HRU and land use name lookup: `hru_read.f90`, `hru_module.f90`, `landuse_read.f90`
- Operation name lookup and execution: `read_mgtops.f90`, `mgt_sched.f90`
- Percolation: `perco` conversion in `topohyd_init.f90`, use in `swr_percmicro.f90`
- Basin and landscape unit weighting: `basin_output.f90`, `lsu_output.f90`

Item	Fortran
`BSN_FRAC`, `RU_FRAC` in `ls_unit.ele`	`lsu_elem%bsn_frac`, `lsu_elem%ru_frac`
`perco`	`hyd_db%perco`, converted to `hru%hyd%perco_lim`
HRU name pointers	`hru_db%dbs` (`type hru_databases`, initialised to 1)

Purpose

The run time of SWAT+ depends on the length of the simulation, the number of objects (HRUs, channels, aquifers, reservoirs), the time step and the amount of output. The disk space a run needs depends almost entirely on the output switched on in `print.prt`. This page covers what can be changed.

One run per processor core

SWAT+ is a serial program: the source has no OpenMP or MPI code, and one run uses one processor core. To use several cores, run several scenarios or calibration samples at the same time, each in a separate copy of the project

folder. Output file names are fixed, so two runs in the same folder overwrite each other's files.

Build type

The executables on the releases page are optimized builds (`Rel` in the file name; see Install). When building from source, set the build type:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
cmake --build build -j 8
```

`-j 8` only speeds up the compilation. Without a build type, CMake adds no optimization flag and the executable name has no `-Rel` or `-Dbg` suffix. A `Debug` build adds run-time checks (array bounds and floating-point traps with gfortran) and runs much slower; use it to track down a crash and switch back afterwards. With gfortran on Windows, the 62.0.1 CMake setup adds the gfortran run-time checks to every build type, including `Release`. See Building from source.

Time step

`step` in `time.sim` sets the number of time steps per day. With a value above 1, precipitation, infiltration, surface runoff and channel and reservoir routing are calculated for every time step, while plant growth, soil water and nutrient cycling still run once a day. Run time grows with the number of steps. Use a sub-daily step only when the question needs it, for example Green-Ampt infiltration or sub-daily peak flows.

Output volume

Each `y` in the daily column of a `print.prt` row writes one row per object per day, so daily output is about 365 times the size of yearly output and 30 times the size of monthly output. Average annual output adds one row per object. Ways to reduce it:

- Switch daily output on only for the rows you will use, and use monthly, yearly or average annual output for the rest.
- Limit the daily rows to the period you need with the print period on line 3 of `print.prt` (`day_start`, `yrc_start`, `day_end`, `yrc_end` and `interval`), and skip warm-up years with `nyskip`. See Print period.
- Leave `csvout` at `n` unless you need the `.csv` files: with `y`, every output file is written twice, once as text and once as `.csv`.
- With gwflow, check the gwflow rows of `print.prt`. Several gwflow output groups are on unless a row switches them off; see gwflow outputs.

`files_out.out` lists every output file a run opened. Check it after a short first run to see what a long run will write.

Carbon output files

The carbon files are the largest outputs, because the per-layer files have a value for every soil layer and variable.

- **Legacy carbon rows.** The `hru_cb` and `hru_cb_vars` rows with `1` in the daily column write per-layer diagnostics for every HRU on every day. The `Ames_sub1` reference dataset has this setting, and a full 46-year run of its 12 HRUs writes about 9 GB, almost all of it these files. Set the daily column of these rows to `n` unless you need the daily layer values. See Legacy CSU carbon outputs.
- **Per-layer carbon rows.** In 62.0.1 the files of the `hru_cb_lyr`, `hru_cb_cpool`, `hru_cb_npool`, `hru_cb_plt`, `hru_cb_flux`, `hru_cb_drv`, `hru_cb_dyn` and `hru_cb_snap` rows repeat their block of HRU rows once per HRU. Their size grows with the square of the number of HRUs: 100 HRUs give 100 times the rows they should. Use yearly or average annual output for these rows in models with many HRUs. See Repeated rows.
- **Number of layers.** The per-layer files have as many columns per variable as the HRU with the most soil layers. An optional `carbon_layers.prt` file sets a smaller number, for example to write only the top three layers. See Layer count file.

Check the free disk space before a long run with daily output (`df -h .` on Linux and macOS). A disk that fills during a run stops it, often without a clear message; see Runs that stop without a message.

Object count and storage

Run time grows roughly in proportion to the number of HRUs, channels, aquifers and routing units. When testing inputs, use a short simulation period or a small part of the watershed, and run the full model once the inputs are settled.

SWAT+ reads its inputs at the start and writes the outputs during the run. With many daily outputs, a project folder on a local disk runs noticeably faster than one on a network share.

Summary

Change	Effect
`CMAKE_BUILD_TYPE=Release`	optimized code; a `Debug` build runs much slower
`step` 0 or 1 in `time.sim`	daily calculation, fastest
daily output only where needed	smaller files, faster runs
print period and `nyskip` in `print.prt`	fewer daily rows
`csvout = n`	half the output files
no `l` in the daily column of `hru_cb` and `hru_cb_vars`	avoids the largest carbon files
yearly or average annual per-layer carbon files	avoids the repeated HRU rows growing with the square of the HRU count
several runs in separate folders	uses several processor cores

Sources

- Build flags: `CMakeLists.txt` and `CMakePresets.json`
- Output switches: `basin_print_codes_read.f90`, file opening in `output_landscape_init.f90`
- Carbon writers and their calls: `command.f90`, `soil_nutcarb_write.f90`, `soil_carbvar_write.f90`, layer count in `carbon_layers_read.f90`